Workload benchmarks for the generated seccomp profiles: what they are and how to reproduce them

The question the benchmarks answer: does an application still work when it runs under the profile SysJava generated for its image, and does it ever need a syscall outside that profile? Each of the 20 images (the ten JDK 8 images of the SysJava vs SWAT4J table and the ten latest images of Table III) gets a workload that exercises what the application is for: a web server serves and deploys, a database writes, queries and drops data, a script interpreter runs a script that touches files, threads and the network.

1. How one validation run works

Mode	Seccomp profile given to docker run	What it shows
base	Docker's default profile	how many operations the workload completes when nothing of ours is in the way
log	the generated profile with defaultAction SCMP_ACT_LOG	nothing is blocked; every syscall outside the profile is written by the kernel to the audit log (auditd: /var/log/audit/audit.log, type=SECCOMP, code=0x7ffc0000) and attributed to the container through its host PIDs (docker top, polled every second)
enforce	the generated profile as published	whether the application works when the profile is enforced

- Each application runs base, log and enforce three times (9 runs per application, 90 per set). Every request or step of the workload is counted as ok or failed.
- **PASS** = in every repetition, enforce completes the same number of operations as base with 0 failures, and the log run records no syscall outside the profile.
- The workload is driven from the host or from an unconfined client container, never from a tool inside the confined container (Groovy and JRuby are batch programs: the workload script is the program).
- **Positive control** (control/positive_control.py): JRuby runs mkdir, write and rename under its log profile with mkdir and rename removed. The harness must report exactly mkdir and rename. It does, so an empty log is evidence and not a broken pipe.
- **Traced pass** (trace_validation.py, JDK 8 set, for the paper's policy-validation table): one more run per application under Docker's default profile while sysdig records every syscall of the container from start-up to the end of the workload. It gives the syscalls the workload actually performed, which of them are outside the profile, and any library or program loaded that start-up discovery did not see.

2. How the benchmarks were built

Starting point. A reviewer-driven to-do item (sysjava-paper/icse27/MR_todo.md): *validate policies under realistic workloads; use existing benchmarks and tests; manual interaction as a fallback.* On 2026-11-11 we weighed three ways to do it:

Option	Decision	Why
The projects' own unit and integration tests	not used	They need Maven/Gradle, test dependencies and the source tree, so they test a build of the source, not the container image the profile was generated for.
Manual interaction (clicking through a UI, typing commands)	fallback only (not needed)	Not repeatable and not countable: no way to show that enforce did exactly what base did.
Scripted workloads built from each application's own tools, bundled example data and documented APIs	used	Repeatable, every operation counted, and it runs against the exact image that was analysed.

Design rules every workload follows:

- **Use the application the way its users do**, through its own interface: HTTP for the web servers, the REST API for Elasticsearch, Solr and OrientDB, CQL and cassandra-stress for Cassandra, zkCli and the four-letter admin words for ZooKeeper, the storm CLI for Storm, and a script for the two interpreters.
- **Cover the life cycle, not only the hot path.** Each workload creates, writes, reads, administers and deletes: war deploy and undeploy, create and drop database, TRUNCATE, backup, optimize / forcemerge, reindex, core unload with delete. These reach the rarer syscalls (rename, rmdir, unlink, fsync, fallocate, chmod) that a read-only load never touches. Example: Solr's core unload with delete is the path that needs rmdir.
- **Enough volume and concurrency to wake the JVM's background work** (GC, thread pools, JIT, file growth, compaction): 3,000 requests on 16 threads, three 60,000-operation stress phases, 40,000 bulk-indexed documents.
- **Drive from outside.** The workload runs on the host or in an unconfined client container, so the confined container runs only the application, and a client tool's syscalls are never counted against its profile.
- **Count every operation** as ok or failed, with the error, so enforce is compared with base number for number.
- **Fixed data and fixed counts** (no time-based loops), so the three repetitions give identical counts.
- **Accepted only when it passes under Docker's default profile.** Each workload was run under base first and debugged until it completed with 0 failures. Every bug found that way was in the workload, not the profile: Solr query encoding and backup location, a Groovy string, a JRuby threading bug, root-owned work directories, the Jetty and Storm readiness checks.

Where each benchmark comes from: what is the application's own, and what we wrote.

App	From the application / image	Written by us
ZooKeeper 3.4.14	zkCli.sh (ships in the image); the four-letter admin commands (built into the server)	the command list: a 400-znode tree, set/get, sequential and ephemeral znodes, delete
Tomcat 7.0	the example webapps in the image's webapps.dist (servlets, JSPs, docs); Tomcat's autodeploy	the request mix, the test war (packed from the image's ROOT app), a 16-thread HTTP driver in Python
Cassandra 3.0.29	cassandra-stress (the standard Cassandra benchmark, ships in the image) and cqlsh	the CQL statements; stress parameters (write, read, mixed, n=60,000)
Jetty 9.4.51	Jetty's hot deployment from webapps/	the test war (same as Tomcat's) and the HTTP driver
Storm 2.4.0	the storm CLI in the image: list and the blobstore commands	the command sequence (5 blobs, each created, listed, read, updated, deleted)
OrientDB 3.2.19	the documented REST API (database, SQL command and query endpoints)	the schema, 1,500 records, edges, queries and updates
Elasticsearch 7.5.2	the REST API: _bulk, _search, aggregations, _forcemerge, _update_by_query, _reindex	the generated documents and the sequence
Solr 8.7.0	the example documents shipped in the image (example/exampledocs); core admin, update, select, optimize and replication (backup) APIs	a 2,000-document JSON bulk per round, the 120 queries, the sequence
Groovy 4.0.0	the interpreter and its standard library (JSON, XML, zip, HttpServer)	scripts/workload.groovy: 46 steps
JRuby 9.4.2.0	the interpreter, Ruby's standard library and Java interop	scripts/workload.rb: 46 steps

What was planned but not used. The 2026-11-11 plan also named off-the-shelf drivers: Rally for Elasticsearch, YCSB for Cassandra and OrientDB, wrk / ab / JMeter for Tomcat and Jetty, and the storm-starter WordCount topology for Storm. The implemented workloads call the same APIs directly instead; the reason was not written down at the time. Only cassandra-stress is a named benchmark tool. For Storm, a running topology executes in supervisor and worker processes, not in the nimbus container that was analysed and confined. If reviewers expect named benchmarks, these drivers can be added as client-side drivers without changing the method or the pass rule.

3. The benchmarks

JDK 8 set (Debian 10, Temurin 8u504; all images tagged <version>-jdk8-buster). Code: run_validation.py, functions wl_<app>; the batch scripts are in scripts/.

App	Image	Started as / ready when	What the benchmark does	Driven from	Ops per run
ZooKeeper 3.4.14	zookeeper:3.4.14-jdk8-buster	default command; ready = log line 'binding to port'	27 four-letter commands (ruok, stat, srvr, conf, cons, envi, mntr, wchs, dump, x3); then 3 rounds of zkCli: create a tree of 400 znodes, set and get them, ls/stat/sync, a sequential and an ephemeral znode, delete all, rmr	host (sockets) + zkCli in a client container	3,951
Tomcat 7.0	tomcat:7.0-jdk8-buster	webapps = the image's webapps.dist without the manager apps; ready = GET /examples/ = 200	3,000 GETs on the example servlets, JSPs and docs (16 threads), 50 POSTs, hot deploy of a war, requests to it, undeploy	host (HTTP)	3,052
Cassandra 3.0.29	cassandra:3.0.29-jdk8-buster	1 GB heap; ready = log line 'Starting listening for CQL clients'	cqlsh: create keyspace and tables, inserts, selects, TRUNCATE, DROP; cassandra-stress write, read and mixed, n=60,000 each. Counted per step (11 per run), each step is thousands of requests	unconfined client container	11
Jetty 9.4.51	jetty:9.4.51-jdk8-buster	a war in webapps/; ready = GET /wl/tomcat.css = 200 (the image has no JSP support)	3,000 GETs on static files of the deployed war, 50 POSTs, war hot deploy and undeploy	host (HTTP)	3,052
Storm 2.4.0	storm:2.4.0-jdk8-buster	'storm nimbus' on network et-jdk8-net with a ZooKeeper container; ready = Thrift port 6627 accepts	storm list x10; blobstore create / list / cat / update / delete of 5 blobs (35 ops)	unconfined client container (storm CLI)	35
OrientDB 3.2.19	orientdb:3.2.19-jdk8-buster	root password set; ready = GET /listDatabases = 200	REST API: create a database, classes and an index, 1,500 inserts, edges, SQL queries, updates, drop the database	host (HTTP)	1,808
Elasticsearch 7.5.2	elasticsearch:7.5.2-jdk8-buster	1 GB heap; ready = GET /_cluster/health = 200	2 indices x 20 bulk requests of 1,000 documents, searches and aggregations, forcemerge, update_by_query, reindex, delete index	host (HTTP)	250
Solr 8.7.0	solr:8.7.0-jdk8-buster	default command; ready = GET /solr/admin/info/system = 200	create a core, 5 rounds of indexing (example XML docs + a JSON bulk), 120 queries, optimize, backup, unload with delete of the index directory	host (HTTP)	199
Groovy 4.0.0	groovy:4.0.0-jdk8-buster	batch: the container runs groovy /wl/workload.groovy (scripts/ mounted read-only)	5 rounds of: write 200 files, read + regex, rename, JSON, XML, zip, 64 tasks on 8 threads, a local HTTP server with 200 requests, delete the tree; then cleanup (46 steps). Prints OPS ok=N fail=N	the container's own program	46
JRuby 9.4.2.0	jruby:9.4.2.0-jdk8-buster	batch: the container runs jruby /wl/workload.rb	the same kinds of steps in Ruby: files, regex, rename, chmod, JSON, zlib, threads, a local TCP server, Java interop, directory delete (46 steps)	the container's own program	46

Latest set: the same benchmarks (EchoTrace_latest/workload_validation/run_validation.py), adapted where the newer version requires it:

App	Image	Difference from the JDK 8 benchmark	Ops per run
ZooKeeper 3.9.5	zookeeper:3.9.5 (JDK 17)	zkCli path of 3.9; deleteall instead of rmr; four-letter words other than srvr answer 'not in the whitelist' (still an answer)	3,951
Tomcat 11.0.22	tomcat:11.0.22 (JDK 25)	Tomcat 10.1+ allows the examples/docs apps from localhost only: the RemoteCIDRValve is removed	3,052
Cassandra 5.0.8	cassandra:5.0.8 (JDK 17)	same	11
Jetty 12.1.10	jetty:12.1.10 (JDK 25)	image as shipped has no deploy module and no webapps: bare server, 3,000 GETs (404), 50 POSTs, HEAD/OPTIONS/PUT/DELETE, 20 keep-alive connections x 50 requests, an oversized header (431); 3,075 ops	3,075
Storm 2.8.8	storm:2.8.8 (JDK 17)	same, on network et-latest-net with zookeeper:3.9.5	35

App	Image	Difference from the JDK 8 benchmark	Ops per run
OrientDB 3.2.53	orientdb:3.2.53 (JDK 8)	same	1,808
Elasticsearch 7.1.0	elasticsearch:7.1.0 (JDK 12)	same	250
Solr 10.0.0	solr:10.0.0 (JDK 25)	Solr 10 runs SolrCloud by default: Collections API (create / delete collection) instead of cores; 194 ops	194
Groovy 5.0.6	groovy:5.0.6 (JDK 21)	same	46
JRuby 10.1.0.0	jruby:10.1.0.0 (JDK 21)	same	46

4. Results with the published profiles

20 of 20 PASS. 180 runs; no failed operation under any enforced profile, and no syscall outside any profile in the 60 log runs. Before the 2026-11-11 analysis fixes the same benchmarks found real misses (libjvm internals, a RocksDB virtual call, runc, a dlsym lookup in JDK 25's libnio that made Tomcat 11 fail 250 of 3,052 requests per enforced run). Those were fixed in the analysis, not by adding syscalls by hand; details in Workload_validation_all20.pdf.

App (JDK 8 / latest version)	JDK 8: profile	ok / failed, base · log · enforce (3 runs each)	Outside profile	Result	Latest: profile	ok / failed, base · log · enforce	Outside profile	Result
ZooKeeper 3.4.14 / 3.9.5	192	11,853 / 0 · 11,853 / 0 · 11,853 / 0	none	PASS	196	11,853 / 0 · 11,853 / 0 · 11,853 / 0	none	PASS
Tomcat 7.0 / 11.0.22	173	9,156 / 0 · 9,156 / 0 · 9,156 / 0	none	PASS	180	9,156 / 0 · 9,156 / 0 · 9,156 / 0	none	PASS
Cassandra 3.0.29 / 5.0.8	230	33 / 0 · 33 / 0 · 33 / 0	none	PASS	233	33 / 0 · 33 / 0 · 33 / 0	none	PASS
Jetty 9.4.51 / 12.1.10	161	9,156 / 0 · 9,156 / 0 · 9,156 / 0	none	PASS	191	9,225 / 0 · 9,225 / 0 · 9,225 / 0	none	PASS
Storm 2.4.0 / 2.8.8	179	105 / 0 · 105 / 0 · 105 / 0	none	PASS	207	105 / 0 · 105 / 0 · 105 / 0	none	PASS
OrientDB 3.2.19 / 3.2.53	174	5,424 / 0 · 5,424 / 0 · 5,424 / 0	none	PASS	183	5,424 / 0 · 5,424 / 0 · 5,424 / 0	none	PASS
Elasticsearch 7.5.2 / 7.1.0	185	750 / 0 · 750 / 0 · 750 / 0	none	PASS	208	750 / 0 · 750 / 0 · 750 / 0	none	PASS
Solr 8.7.0 / 10.0.0	201	597 / 0 · 597 / 0 · 597 / 0	none	PASS	224	582 / 0 · 582 / 0 · 582 / 0	none	PASS
Groovy 4.0.0 / 5.0.6	189	138 / 0 · 138 / 0 · 138 / 0	none	PASS	191	138 / 0 · 138 / 0 · 138 / 0	none	PASS
JRuby 9.4.2.0 / 10.1.0.0	168	138 / 0 · 138 / 0 · 138 / 0	none	PASS	187	138 / 0 · 138 / 0 · 138 / 0	none	PASS

Traced pass, JDK 8 set (paper table: policy validation)

App	Workload ops	Missing	Performed	Permitted	New binaries	New libraries	Loaded under another name of an analysed library
ZooKeeper 3.4.14	3,951	0	85	192	0	0	-
Tomcat 7.0	3,052	0	82	173	0	0	-
Cassandra 3.0.29	11	0	104	230	0	0	/tmp/libnetty-transport-native-epoll8125506845405275135.so = libnetty-transport-native-epoll.so; /usr/local/lib/libjemalloc.so = libjemalloc.so.2
Jetty 9.4.51	3,052	0	67	161	0	0	-
Storm 2.4.0	35	0	93	179	0	0	-
OrientDB 3.2.19	1,808	0	79	174	0	0	/tmp/jffi2757544223715092008.so = libjffi-1.2.so

App	Workload ops	Missing	Performed	Permitted	New binaries	New libraries	Loaded under another name of an analysed library
Elasticsearch 7.5.2	250	0	88	185	0	0	-
Solr 8.7.0	199	0	86	201	0	0	-
Groovy 4.0.0	46	0	79	189	0	0	-
JRuby 9.4.2.0	46	0	78	168	0	0	-

Missing = performed syscalls not in the profile. Performed excludes runc's setup stages (runc:[0:PARENT], runc:[1:CHILD]), which run before the seccomp filter is installed; they are listed per app in table_row.json (runc_setup_syscalls). New = an executed program or opened .so whose name is not in the start-up dataset (Dataset/LIBS_<t>, BINARIES_<t>). Each workload uses roughly half of its profile.

5. How to reproduce

5.1 What you need

- Linux x86-64 with Docker (we used Docker 29.1.3, kernel 5.15) and passwordless sudo for reading the audit log and for sysdig.
- **auditd running**: with auditd, SCMP_ACT_LOG records go to /var/log/audit/audit.log, not to the kernel log. Run the positive control first; if it does not report mkdir and rename, log mode is not seeing anything.
- Python 3 (the harness uses only the standard library); reportlab only for the PDFs.
- sysdig 0.41.4 with --modern-bpf, only for the traced pass.
- The 20 images pulled (the same images the pipeline analysed), and about 120 GB of RAM if three groups run in parallel (Elasticsearch, Cassandra and Solr each take 1-2 GB heap plus clients).
- The profiles. In this workspace they are read from the pipeline results (Final_results_JDK8/<t>/Binary_Analysis/syscalls_output_<t>/<t>.json); in the artifact (analysis/workload_validation/) the harness falls back to profiles/fixed/ next to it. WLV_PROFILE_DIR points it at any folder of <t>.json.

5.2 JDK 8 set: the validation (90 runs)

```
cd ~/Tutorial/EchoTrace/workload_validation            # artifact: analysis/workload_validation
python3 -c "import run_validation as r; r.prepare()"   # once: Solr example docs, Tomcat webapps, the test war
python3 control/positive_control.py control/result_check.json   # must print exactly mkdir and rename
export WLV_RESULTS=results_fixes
export WLV_STORM_PROFILE=~/Tutorial/EchoTrace/Final_results_JDK8/_tables/storm_no_false_ffm_fixes/storm_2.4.0-jdk8-buster.json
python3 run_validation.py tomcat jetty storm --reps 3 &
python3 run_validation.py elasticsearch solr cassandra --reps 3 &
python3 run_validation.py zookeeper orientdb groovy jruby --reps 3 &
wait
python3 make_table.py                                  # -> results_fixes/table.md
```

- Output per run: results_fixes/<app>/<mode>_<rep>.json (ok, failed, would_be_denied), .container.log, and for log mode .audit.log (the raw records); results_fixes/summary.tsv for all runs.
- Expected: every row PASS; base, log and enforce ops identical (e.g. Tomcat 3,052, ZooKeeper 3,951, OrientDB 1,808 per run); would_be_denied empty everywhere.
- Time: each application takes minutes (Storm and Cassandra the longest, about 15-20 min for 9 runs, because every storm and cassandra-stress command starts a JVM); the three groups together take well under an hour.
- Storm needs WLV_STORM_PROFILE: without it the harness uses the older 172-syscall profile, and log mode then reports fallocate and fchdir (the two syscalls the fixes added).

5.3 JDK 8 set: the traced pass (policy-validation table)

```
cd ~/Tutorial/EchoTrace/workload_validation
python3 trace_validation.py zookeeper tomcat cassandra jetty storm orientdb elasticsearch solr groovy jruby
python3 trace_validation.py --analyse <apps>           # optional: recompute from the saved traces
python3 results_trace/make_table.py                    # -> results_trace/table.md, policy_validation_rows.tex
```

- trace_validation.py already uses the 179 Storm profile and writes to results_trace/.
- Output: results_trace/<app>/trace_1.sysdig.txt (de-duplicated sysdig events), trace_1.json, and table_row.json (missing, performed, permitted, new libraries/binaries and the lists behind each number).

- Expected: 0 missing and 0 / 0 new for all ten; performed 85 82 104 67 93 79 88 86 79 78 (ZooKeeper, Tomcat, Cassandra, Jetty, Storm, OrientDB, Elasticsearch, Solr, Groovy, JRuby).

5.4 Latest set (90 runs)

```
cd ~/Tutorial/EchoTrace_latest/workload_validation     # artifact: analysis/workload_validation_latest
python3 -c "import run_validation as r; r.prepare()"
python3 control/positive_control.py control/result_check.json
export WLV_RESULTS=results_fixes
export WLV_STORM_PROFILE=~/Tutorial/EchoTrace_latest/unavailable_libs/storm_2.8.8/merged_rta_fixes/storm_2.8.8_rta_with_supplement_no_sync.json
python3 run_validation.py tomcat jetty storm --reps 3 &
python3 run_validation.py elasticsearch solr cassandra --reps 3 &
python3 run_validation.py zookeeper orientdb groovy jruby --reps 3 &
wait
python3 make_table.py
```

5.5 The PDFs

```
cd ~/Tutorial/EchoTrace/workload_validation
<venv>/bin/python make_pdf_all20.py        # Workload_validation_all20.pdf (both sets, before/after the fixes)
<venv>/bin/python make_pdf_benchmarks.py   # this document
```

5.6 Pitfalls we hit (so you don't)

- **Reading the wrong log.** With auditd running, SCMP_ACT_LOG records never reach journalctl -k. Our first 30 log runs read the journal and reported nothing; the positive control exposed it. The harness reads /var/log/audit/audit.log.
- **Silence is not proof.** auditd can stop writing other events (audit_enabled=0) while SECCOMP records still arrive. Run the positive control in the same session as the benchmarks.
- **Do not start several runs before prepare().** The parallel groups would each try to copy the Solr docs and Tomcat webapps at once.
- **Harness failures look like profile failures.** A wrong readiness URL or a bug in a workload fails under base too. Always compare enforce against base, never against the expected count alone.
- **Root-owned work directories.** Tomcat and Jetty run as root in the container and leave root-owned files in work/; the harness removes them with sudo before each run.
- **sysdig (traced pass).** Use a display filter (-d), not a capture filter: with a capture filter sysdig only enables the syscalls it thinks the filter needs. Start the format with * or events without an fd are dropped. Newer sysdig reports many syscalls as exit events only, so read syscall.type, not the enter event. sysdig names pread, pwrite and prlimit differ from libseccomp's (pread64, pwrite64, prlimit64).
- **runc's setup stages** appear in the container's trace but run before the filter is installed; count them separately, as the traced pass does.
- **Memory.** Running all ten applications at once with three JVM-heavy groups needs about 120 GB; run fewer groups in parallel on a smaller host.

6. Where everything is

```
JDK 8:   ~/Tutorial/EchoTrace/workload_validation/
           run_validation.py  trace_validation.py  make_table.py  scripts/ (batch workloads)  control/ (positive control)
           results_fixes/ (published profiles)  results/ (before the fixes)  results_trace/ (traced pass)
           REPRODUCE.md (full log of what was run and observed)
latest:  ~/Tutorial/EchoTrace_latest/workload_validation/  (same layout; results_current/ = before the fixes)
Artifact: analysis/workload_validation/ and analysis/workload_validation_latest/ (pack_artifact.sh in each folder)
```